

Visualizing Tech Stocks

This analysis will review 5 technology stocks (Microsoft (MSFT), Amazon (AMZN), Apple (AAPL), Google (GOOGL), and Meta (META)) over a six month period (January 2023 through June 2023). The libraries used will be pandas, numpy, yfinance, and Matplotlib.

```
In [3]: # Importing libraries
# %matplotlib inline requires Jupyter Notebook to plot the chart below the line of code instead of a p

import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [8]: # creating variables to be used to pull the required stock data and pulling the data

symbols = ["MSFT", "AMZN", "AAPL", "GOOGL", "META"]
start_date = "2023-01-01"
end_date = "2023-07-01"
stock_data = yf.download(symbols, start=start_date, end=end_date, auto_adjust=False)
stock_data
```

[*****100%*****] 5 of 5 completed

Out[8]:

Price		Adj Close								
Ticker		AAPL	AMZN	GOOGL	META	MSFT	AAPL	AMZN	GOOGL	M
Date										
2023-01-03		122.982735	85.820000	88.336700	123.654129	232.948242	125.070000	85.820000	89.120003	124.739
2023-01-04		124.251198	85.139999	87.305840	126.261238	222.758377	126.360001	85.139999	88.080002	127.370
2023-01-05		122.933556	83.120003	85.442360	125.834969	216.156296	125.019997	83.120003	86.199997	126.940
2023-01-06		127.456779	86.080002	86.572334	128.888184	218.703781	129.619995	86.080002	87.339996	130.020
2023-01-09		127.977928	87.360001	87.246361	128.342941	220.833160	130.149994	87.360001	88.019997	129.470
...		...	...	...	...	...	...	...	...	...
2023-06-26		182.708572	127.330002	117.299873	276.045898	321.004608	185.270004	127.330002	118.339996	278.470
2023-06-27		185.459991	129.179993	117.289970	284.551178	326.836578	188.059998	129.179993	118.330002	287.049
2023-06-28		186.633591	129.039993	119.123695	282.806549	328.087067	189.250000	129.039993	120.180000	285.290
2023-06-29		186.968842	127.900002	118.053192	279.079254	327.305450	189.589996	127.900002	119.099998	281.529
2023-06-30		191.288254	130.360001	118.647919	284.481812	332.668640	193.970001	130.360001	119.699997	286.980

124 rows x 30 columns

```
In [10]: # Isolating the data for adjusted close price to perform the analysis on this datapoint.
```

```
adj_close = stock_data['Adj Close']  
adj_close
```

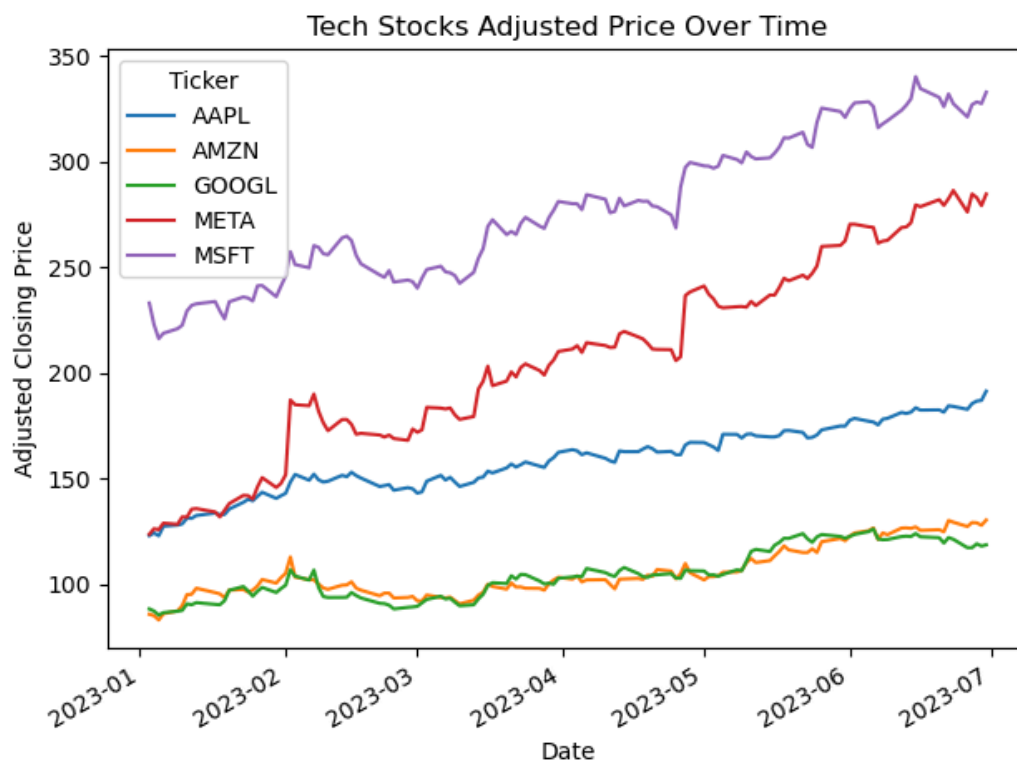
```
Out[10]:
```

	Ticker	AAPL	AMZN	GOOGL	META	MSFT
	Date					
	2023-01-03	122.982735	85.820000	88.336700	123.654129	232.948242
	2023-01-04	124.251198	85.139999	87.305840	126.261238	222.758377
	2023-01-05	122.933556	83.120003	85.442360	125.834969	216.156296
	2023-01-06	127.456779	86.080002	86.572334	128.888184	218.703781
	2023-01-09	127.977928	87.360001	87.246361	128.342941	220.833160
	...	...	...	...	...	...
	2023-06-26	182.708572	127.330002	117.299873	276.045898	321.004608
	2023-06-27	185.459991	129.179993	117.289970	284.551178	326.836578
	2023-06-28	186.633591	129.039993	119.123695	282.806549	328.087067
	2023-06-29	186.968842	127.900002	118.053192	279.079254	327.305450
	2023-06-30	191.288254	130.360001	118.647919	284.481812	332.668640

124 rows x 5 columns

```
In [11]: # Plotting the adjusted closing prices over time
```

```
adj_close.plot()  
plt.xlabel('Date')  
plt.ylabel('Adjusted Closing Price')  
plt.title('Tech Stocks Adjusted Price Over Time')  
plt.tight_layout()  
plt.show()
```



```
In [13]: # Calculating and plotting the daily simple rate of return over time
```

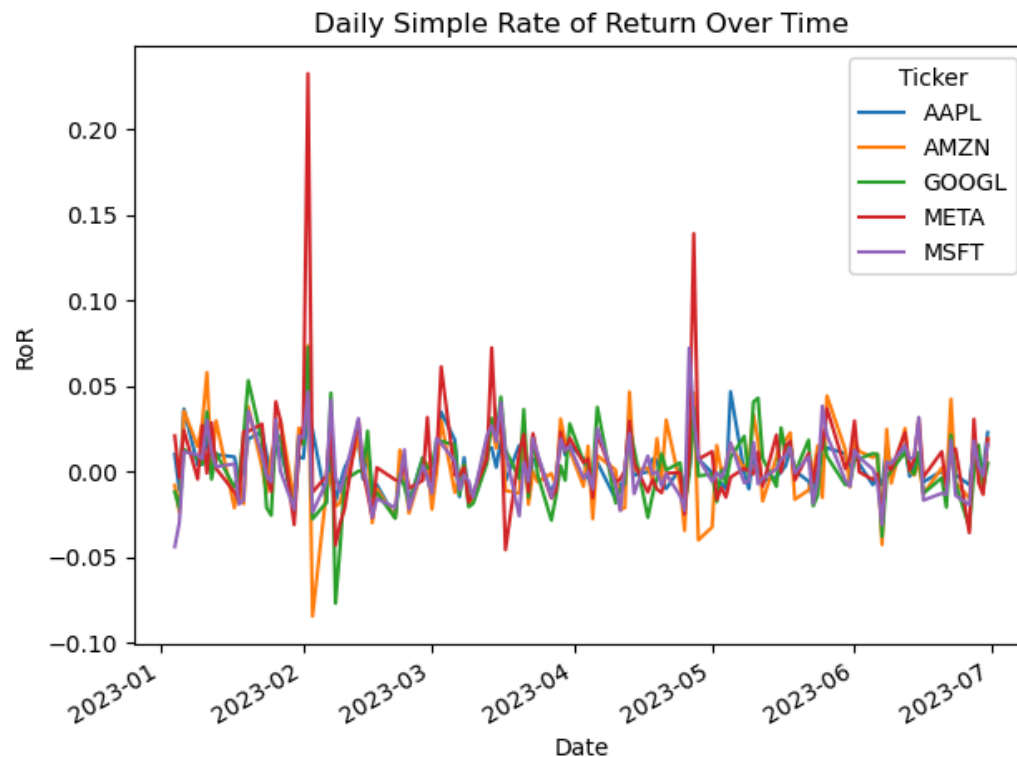
```

stock_data_daily_returns = adj_close.pct_change()

plt.figure(figsize=(16,9))
stock_data_daily_returns.plot()
plt.xlabel('Date')
plt.ylabel('RoR')
plt.title('Daily Simple Rate of Return Over Time')
plt.tight_layout()
plt.show()

```

<Figure size 1600x900 with 0 Axes>

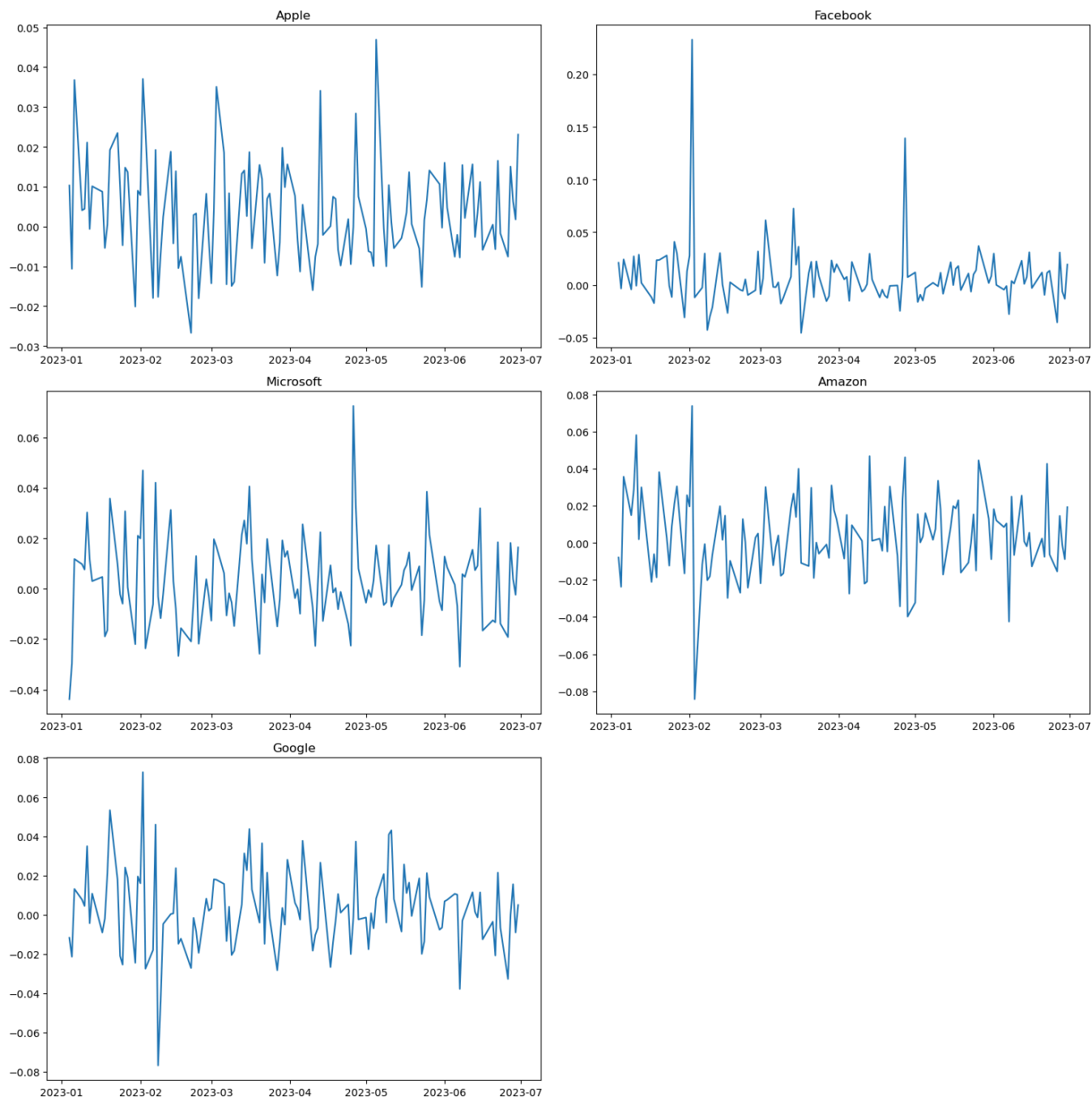


In [15]: # Creating multiple individual plots for their simple rate of return review. This makes reviewing ind

```

fig = plt.figure(figsize=(15, 15))
ax1 = fig.add_subplot(321)
ax2 = fig.add_subplot(322)
ax3 = fig.add_subplot(323)
ax4 = fig.add_subplot(324)
ax5 = fig.add_subplot(325)
ax1.plot(adj_close["AAPL"].pct_change())
ax1.set_title("Apple")
ax2.plot(adj_close["META"].pct_change())
ax2.set_title("Facebook")
ax3.plot(adj_close["MSFT"].pct_change())
ax3.set_title("Microsoft")
ax4.plot(adj_close["AMZN"].pct_change())
ax4.set_title("Amazon")
ax5.plot(adj_close["GOOGL"].pct_change())
ax5.set_title("Google")
plt.tight_layout()
plt.show()

```



In [16]: *# Calculating the mean of each tech stock's daily simple rate of return*

```
daily_mean = stock_data_daily_returns.mean()
daily_mean
```

Out[16]: Ticker
 AAPL 0.003682
 AMZN 0.003652
 GOOGL 0.002619
 META 0.007224
 MSFT 0.003060
 dtype: float64

In [17]: *# Getting the indexes for the x-axis*

```
daily_mean.keys()
```

Out[17]: Index(['AAPL', 'AMZN', 'GOOGL', 'META', 'MSFT'], dtype='object', name='Ticker')

In [18]: *# Getting the daily means for each stock for the y-axis*

```
height = []
for key in daily_mean.keys():
    height.append(daily_mean[key])
height
```

```
Out[18]: [np.float64(0.003681780564754789),
 np.float64(0.0036520392611041766),
 np.float64(0.002619028428549065),
 np.float64(0.0072235361093724),
 np.float64(0.0030603273163286194)]
```

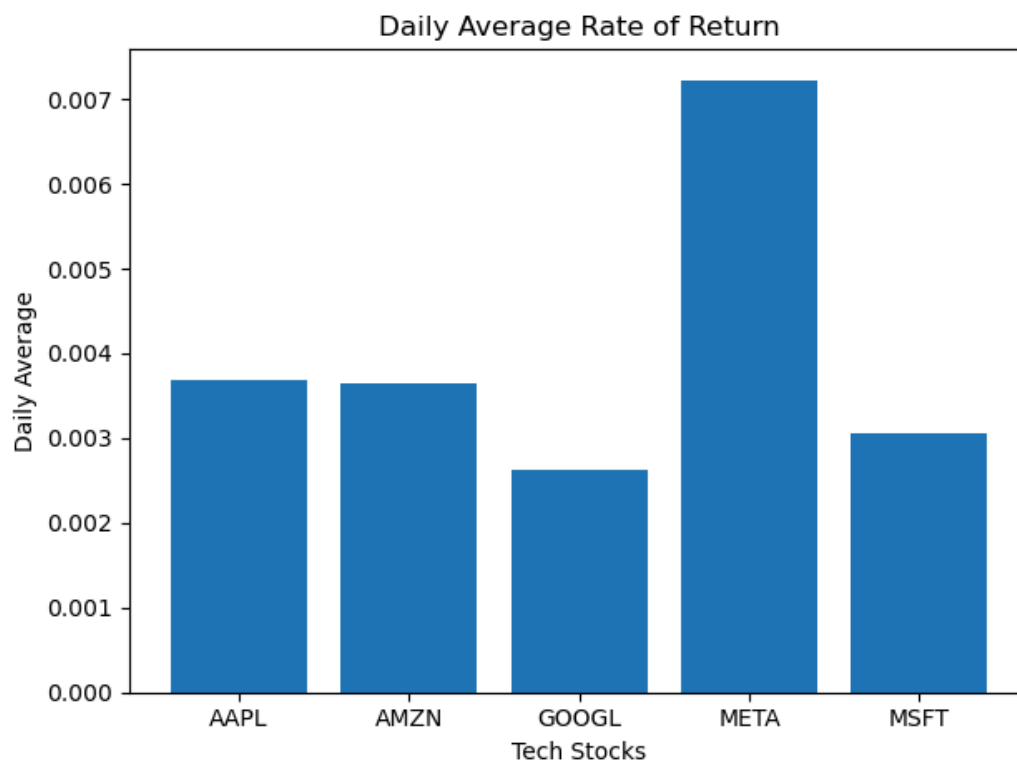
```
In [20]: # Arranging the keys on the x-axis based on the length
```

```
x_pos = np.arange(len(daily_mean.keys()))
x_pos
```

```
Out[20]: array([0, 1, 2, 3, 4])
```

```
In [21]: # Plotting the means of each stock
```

```
plt.bar(x_pos, height)
plt.xticks(x_pos, daily_mean.keys())
plt.xlabel('Tech Stocks')
plt.ylabel('Daily Average')
plt.title('Daily Average Rate of Return')
plt.tight_layout()
plt.show()
```



Based on the visual above, Meta had the highest daily average rate of return. If an investor is looking for higher rates of return, this stock would have been a good stock to invest in during the time period (first half of 2023).

```
In [22]: # Calculating the variance for each stock
```

```
daily_var = stock_data_daily_returns.var()
daily_var
```

```
Out[22]: Ticker
AAPL      0.000171
AMZN      0.000500
GOOGL     0.000440
META      0.000931
MSFT      0.000324
dtype: float64
```

```
In [23]: # Getting the index for the x-axis

daily_var.keys()
```

```
Out[23]: Index(['AAPL', 'AMZN', 'GOOGL', 'META', 'MSFT'], dtype='object', name='Ticker')
```

```
In [24]: # getting the individual variances to plot for comparison

height = []
for key in daily_var.keys():
    height.append(daily_var[key])
height
```

```
Out[24]: [np.float64(0.0001706899801534664),
 np.float64(0.000499641751101315),
 np.float64(0.0004400436990818194),
 np.float64(0.0009308594501412479),
 np.float64(0.00032376346607544335)]
```

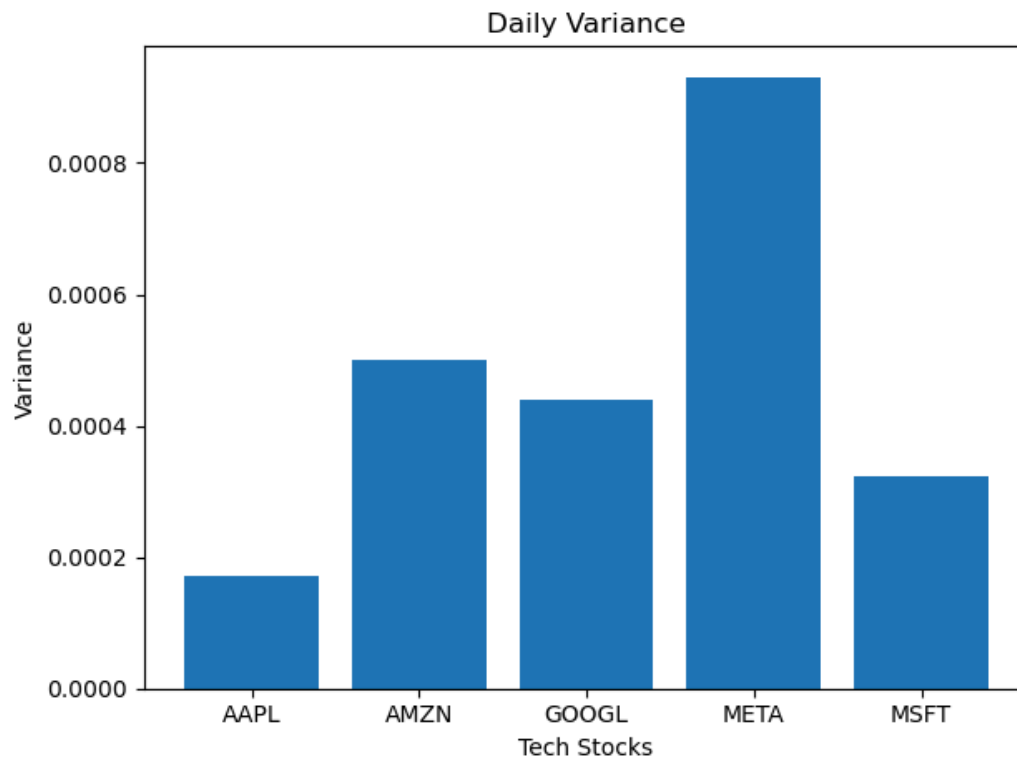
```
In [25]: # Arranging keys on the x-axis based on the length

x_pos = np.arange(len(daily_var.keys()))
x_pos
```

```
Out[25]: array([0, 1, 2, 3, 4])
```

```
In [26]: # Plotting the variance for each stock

plt.bar(x_pos, height)
plt.xticks(x_pos, daily_var.keys())
plt.xlabel('Tech Stocks')
plt.ylabel('Variance')
plt.title('Daily Variance')
plt.tight_layout()
plt.show()
```



Based on the above visual, the stock with the least variance was Apple. If an investor wanted a predictable return, Apple would have been a good choice at that time. On the other hand, Meta had the largest variance, displaying that it could be a riskier stock to invest in at that time.

In [27]: *# Calculating the standard deviation*

```
daily_std = stock_data_daily_returns.std()
daily_std
```

Out[27]: Ticker

```
AAPL    0.013065
AMZN    0.022353
GOOGL   0.020977
META    0.030510
MSFT    0.017993
dtype: float64
```

In [28]: *# Getting the indexes for the x-axis to plot the data*

```
daily_std.keys()
```

Out[28]: Index(['AAPL', 'AMZN', 'GOOGL', 'META', 'MSFT'], dtype='object', name='Ticker')

In [29]: *# getting the bar heights for the standard deviation*

```
height = []
for key in daily_std.keys():
    height.append(daily_std[key])
height
```

Out[29]: [np.float64(0.013064837547917173),
np.float64(0.022352667650670132),
np.float64(0.02097721857353399),
np.float64(0.030509989350067757),
np.float64(0.017993428413602654)]

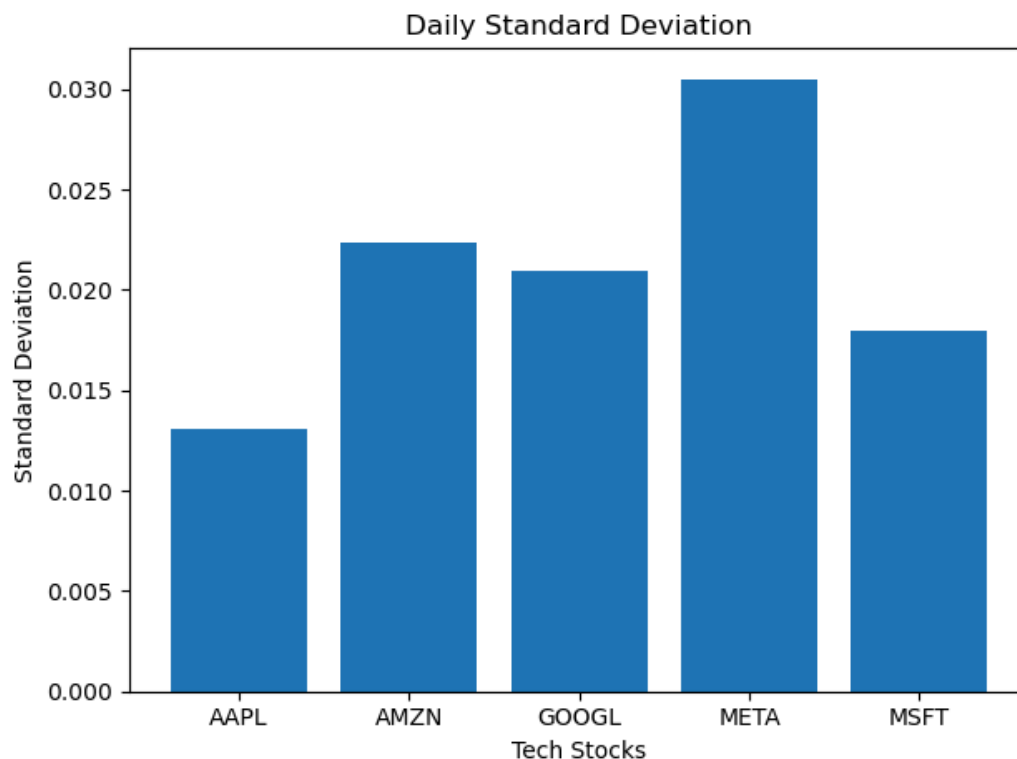
In [30]: *# Arranging the keys on the x-axis based on the length*

```
x_pos = np.arange(len(daily_std.keys()))
x_pos
```

```
Out[30]: array([0, 1, 2, 3, 4])
```

```
In [32]: # Plotting the standard deviations

plt.bar(x_pos, height)
plt.xticks(x_pos, daily_std.keys())
plt.xlabel('Tech Stocks')
plt.ylabel('Standard Deviation')
plt.title('Daily Standard Deviation')
plt.tight_layout()
plt.show()
```



Based on the above visual, Meta has the highest standard deviation, indicating it is more volatile than the other stocks. On the other hand, Apple has the lowest standard deviation indicating less volatility, a more predictable return stock.

```
In [33]: # Calculating the correlations

stock_data_daily_returns.corr()
```

```
Out[33]:
```

Ticker	AAPL	AMZN	GOOGL	META	MSFT
Ticker					
AAPL	1.000000	0.555240	0.599271	0.578897	0.583447
AMZN	0.555240	1.000000	0.630501	0.578416	0.644661
GOOGL	0.599271	0.630501	1.000000	0.621096	0.628549
META	0.578897	0.578416	0.621096	1.000000	0.533363
MSFT	0.583447	0.644661	0.628549	0.533363	1.000000

None of the stocks are negatively correlated, and should respond somewhat similarly to external economic factors. All stocks are relatively close to each other regarding correlation, but some show a higher degree of correlation (such as Amazon and Microsoft, and Microsoft and Google) than others.